

Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
«НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ
ИТМО» (УНИВЕРСИТЕТ ИТМО)
Факультет «Систем управления и робототехники»

Частотные методы

Отчёт по лабораторной работе №6

Работу
выполнил:
Сухов Н. М.
Группа: R3235
Преподаватель:
Догадин Е. В.

Санкт-Петербург
2025

Содержание

1. Фильтрация изображений с периодичностью	3
2. Исследование свёртки	5
3. Приложение	7

1. Фильтрация изображений с периодичностью

Скачаем изображение 1 с [этого гугл-диска](#) (см. рис. 1.1). Видим периодичность с углом около 40° .

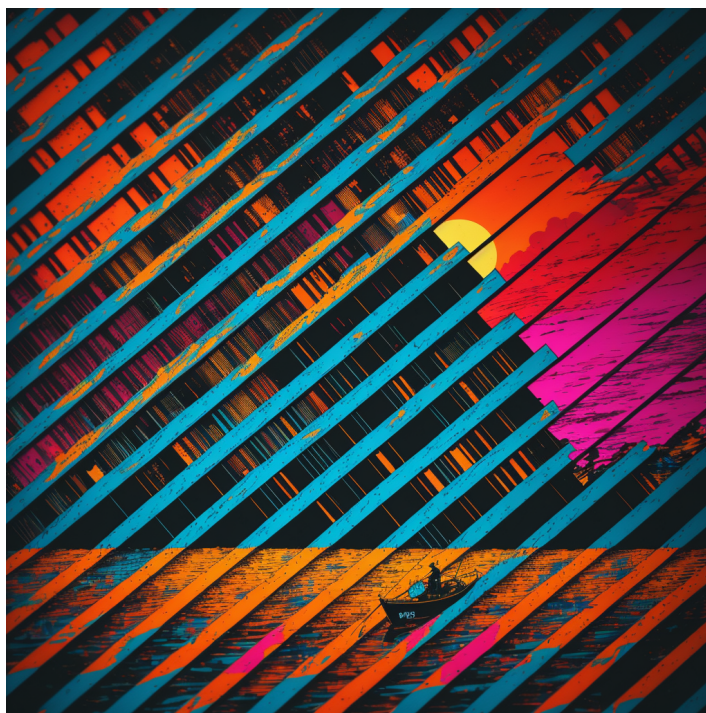


Рисунок 1.1. Картинка

Выполним следующие шаги:

- Загрузим изображение в Python при помощи команды `cv2.imread()`.
- Преобразуем полученный массив к вещественному типу и поделим все значения на 255 - максимальное значение яркости цвета.
- Найдём двумерный Фурье-образ массива и сдвинем его в центр с помощью команд `fft.fftshift(fft.fft2())` для каждого из цветовых каналов.
- Разделим полученный образ на массивы модулей и аргументов с помощью функций `np.abs` и `np.angle`.
- Для удобства работы, найдём логарифм от массива модулей и нормализуем его значения в диапазон от 0 до 1. Чтобы избежать неопределённости в логарифме, предварительно прибавим ко всем значениям 1.
- Сохраним полученный массив (нормализованный логарифм модуля Фурье-образа) как изображение командой `cv2.imwrite`.
- Проанализируем полученное на предыдущем шаге изображение. Найдём пики, соответствующие периодичности на исходной картинке.
- Отредактируем полученный Фурье-образ: сгладим все ненужные цветовые пики, отвечающие за гармоники, от которых хотим избавиться.
- Восстановим картинку из отредактированного образа, проделав обратные шаги.

Изображение логарифма модуля образа исходной картинке представлено на рис. 1.2.

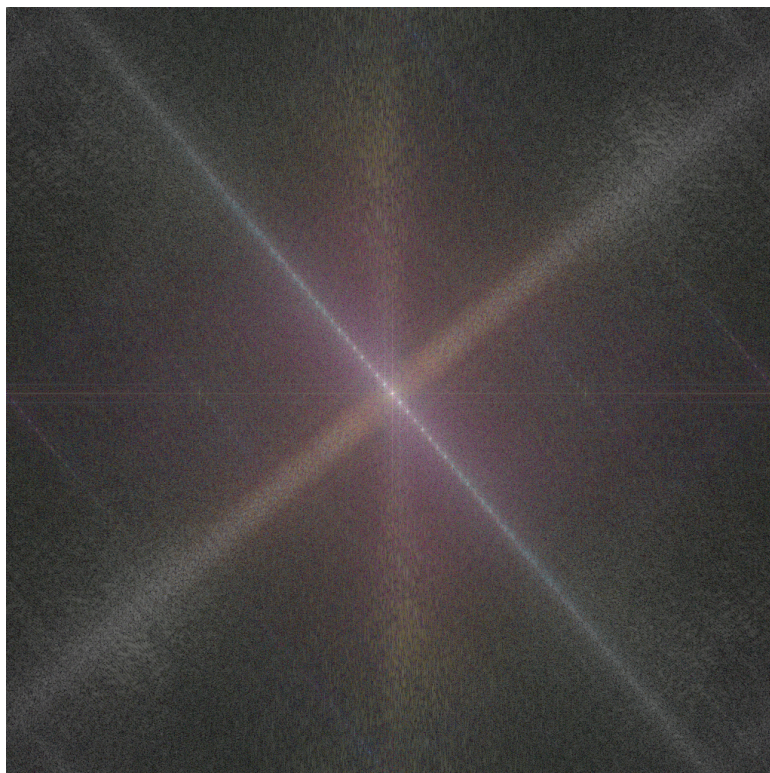
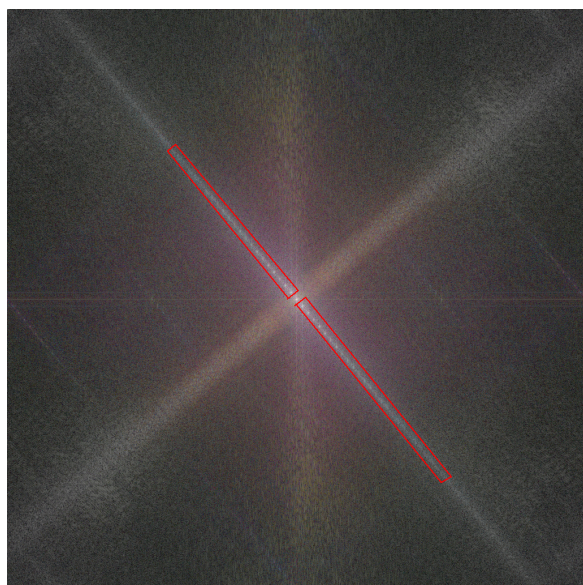
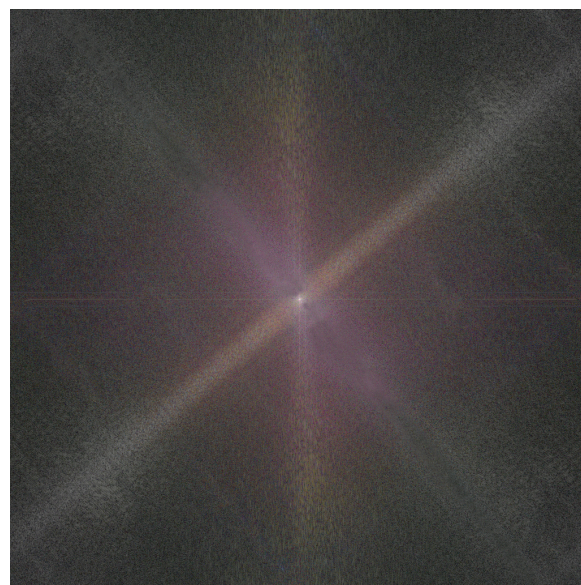


Рисунок 1.2. Логарифм модуля образа

Мы видим множество пиков, расположенных по диагонали на картинке. Выделим их, уберём и посмотрим, что получится (см. рис.1.3).



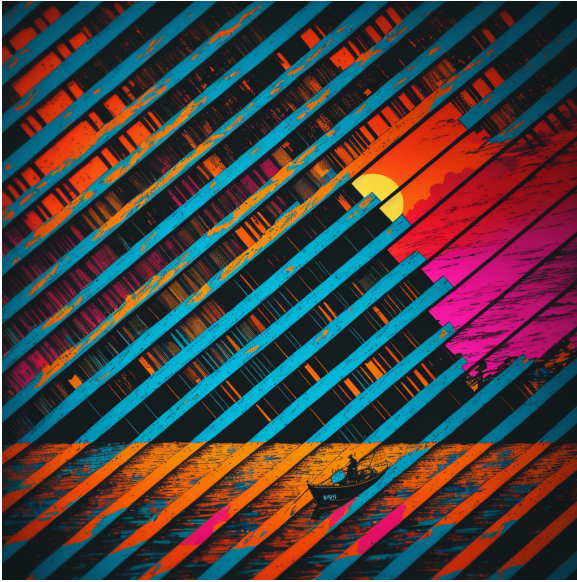
(a)



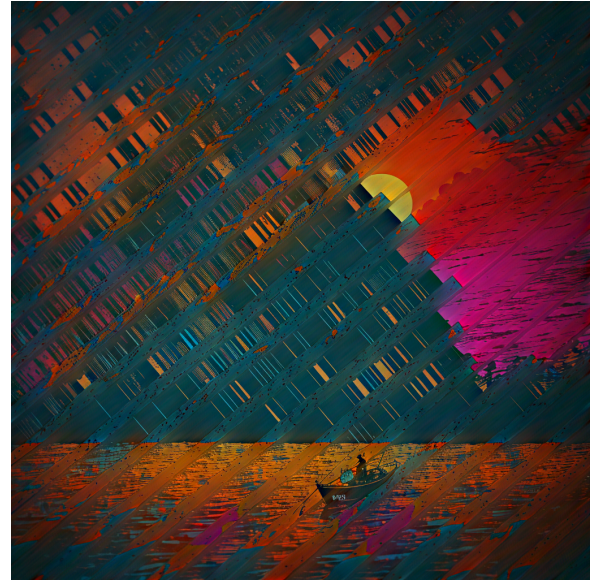
(b)

Рисунок 1.3. Выделенные области с пиками на спектре (a) и очищенный спектр (b)

Сравним теперь исходное и отфильтрованное изображение (см. рис. 1.4). Видим, что периодичность исчезла почти полностью, но есть и побочный эффект - изображение стало более тусклым. Это происходит потому что, затирая пики, мы задеваем часть остальной информации изображения, от чего оно искажается.



(a)



(b)

Рисунок 1.4. Исходное изображение (a) и отфильтрованное (b)

Выводы

Используя многомерное преобразование Фурье, можно осуществлять фильтрацию на изображениях, в данном случае, мы видим, как можно убирать гармоники из спектра, создающие периодические структуры на изображении. При этом чем сильнее мы гасим пики на спектре, тем больше информации мы задеваем, и изображение немного искажается (меняются оттенки), поэтому нельзя затирать их слишком сильно, нужно искать "золотую середину", когда периодичность убрана достаточно хорошо, и при этом нет сильного изменения цветов на изображении, чтобы оно не стало совсем тусклым.

2. Исследование свёртки

Выберем изображение не очень большого размера (см. рис. 2.1a). Преобразуем его в ЧБ (см. рис. 2.1b).

Выберем ядра свёртки:

- Ядра размытия по Гауссу для трёх показательных нечётных значений $N \geq 3$ (выберем N из 3, 7, 15):

$$\sigma = \frac{N-1}{6}, \quad A_{ij} = \exp\left(-\frac{(i - \frac{N+1}{2})^2 + (j - \frac{N+1}{2})^2}{2\sigma^2}\right), \quad i, j \in \{1, \dots, N\},$$

$$K_\sigma = \frac{A}{\sum_{i,j} A_{i,j}}, \quad \text{где } A - \text{вся матрица}$$

- Ядра блочного размытия для тех же значений N :

$$A_{ij} = 1, \quad i, j \in \{1, \dots, N\}, \quad K_\square = \frac{A}{\sum_{i,j} A_{i,j}}$$

- Ядро увеличения резкости:

$$K_* = \begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

- Ядро выделения краёв:

$$K_{\nabla} = \begin{bmatrix} -1 & -1 & -1 \\ -1 & 8 & -1 \\ -1 & -1 & -1 \end{bmatrix}$$

- Ядро негатива:

$$K_{neg} = \begin{bmatrix} 0 & 0 & 0 \\ 0 & -1 & 0 \\ 0 & 0 & 0 \end{bmatrix}$$



(a)



(b)

Рисунок 2.1. Выбранное изображение (a) и ЧБ (b)

3. Приложение

```
1  # %%
2  import numpy as np
3  import cv2
4  from scipy import fft
5
6  # %%
7  plt_folder = "plots/"
8
9  # %% [markdown]
10 # # Задание 1
11
12 # %%
13 image_float = cv2.imread('1.png').astype(np.float64) / 255.0
14
15 fft_channels = []
16 for channel in range(3):
17     channel_data = image_float[:, :, channel]
18     fft_channel = fft.fftshift(fft.fft2(channel_data))
19     fft_channels.append(fft_channel)
20 fft_image = np.stack(fft_channels, axis=-1)
21
22 original_fft = fft_image.copy()
23
24 magnitude_spectrum = np.abs(fft_image)
25 phase_spectrum = np.angle(fft_image)
26
27 log_magnitude = np.log1p(magnitude_spectrum)
28 normalized_log = (log_magnitude - np.min(log_magnitude)) /
29     ↳ (np.max(log_magnitude) - np.min(log_magnitude))
30
31 spectrum_image = (normalized_log * 255).astype(np.uint8)
32 cv2.imwrite(plt_folder+'fourier_spectrum_1.png', spectrum_image)
33
34 # %%
35 spectrum_filtered = cv2.imread(plt_folder +
36     ↳ 'fourier_spectrum_1_filtered.jpg').astype(np.float64) / 255.0
37
38 log_magnitude_filtered = spectrum_filtered * (np.max(log_magnitude) -
39     ↳ np.min(log_magnitude)) + np.min(log_magnitude)
40 magnitude_filtered = np.expml(log_magnitude_filtered)
41
42 filtered_fft_channels = []
43 for channel in range(3):
44     magnitude_channel = magnitude_filtered[:, :, channel]
45     phase_channel = phase_spectrum[:, :, channel]
46     complex_spectrum = magnitude_channel * np.exp(1j * phase_channel)
47     ↳ filtered_fft_channels.append(fft.ifft2(fft.ifftshift(complex_spectrum)))
48
49 reconstructed_image = np.stack(filtered_fft_channels, axis=-1)
50 reconstructed_image = np.real(reconstructed_image)
51 reconstructed_image = np.clip(reconstructed_image, 0, 1)
52 reconstructed_image_uint8 = (reconstructed_image * 255).astype(np.uint8)
53 cv2.imwrite(plt_folder + 'periodic_filtered_image.png',
54     ↳ reconstructed_image_uint8)
```

Листинг 1: Листинг